VelozMart MPS

Marketplace Priority Sorter · Especificação do projeto, decisões do grupo e plano de testes · Coding Tank, Ordenação e Prioridade · 10/11/2026

Resumo. O algoritmo ainda não foi escolhido. Há dois finalistas: **EDF** (menor prazo absoluto primeiro) e **prazo virtual** (prazo absoluto menos um bônus proporcional ao score). Ambos usam uma chave estática, então a estrutura é a mesma: **heap como fila de prioridade** com mapa id→posição, um heap por tipo de doca. Os pedidos são roteados para **docas especializadas por tamanho** (Express, Padrão e Pesada). Cinco outras estratégias (FIFO, peso composto, duas filas, faixas+aging e WSPT) entram na simulação como referência. A escolha final sai da simulação, com um critério de decisão definido antes dos testes.

Mudança de rumo. A versão anterior escolheu o prazo virtual principalmente para proteger pedidos VIP. O grupo concluiu que proteger VIP **não deve pesar tanto** na escolha. Por isso o EDF, mais simples e com cauda de atraso menor, volta a ser candidato forte. O prazo virtual continua na disputa e agora precisa justificar sua complexidade extra.

1. Problema e enunciado

O VelozMart, grande empresa de logística e varejo da América Latina, exibe em tempo real a lista de pedidos aguardando expedição. A ordem deve minimizar atrasos e custos logísticos. Cada pedido tem três atributos:

Atributo	Descrição
`priorityScore` (0 a 100)	Calculado por IA a partir de distância, SLA, valor do item e reputação do vendedor.
`dispatchWindow` (min)	Minutos restantes até o prazo de expedição expirar.
`sizeCategory` (P, M, G)	Define o espaço ocupado na doca de embalagem e agora também o tipo de doca.

Perguntas do time de logística

1. **I.** Como ordenar pedidos considerando múltiplos critérios?
2. **II.** Como re-enfileirar à medida que novos pedidos chegam ou prazos mudam?
3. **III.** Quais métricas comprovam que a estratégia é melhor que a ordenação simples por tempo?

Requisitos mínimos da entrega (Canvas em PDF)

- Descrição textual do algoritmo ou estrutura (pseudocódigo opcional).
- Diagrama de inclusão, remoção e re-priorização, com legenda e alto contraste.
- Justificativa dos critérios e efeitos sobre métricas (atraso médio, throughput por hora).
- Mapa de trade-offs (simplicidade, desempenho, manutenibilidade), com 3 prós e 3 contras.
- Seções do template: Problema, Algoritmo/Estrutura, Diagrama, Trade-offs, Próximos passos.
- Bônus: Big O, comparação entre abordagens, proposta de A/B ou simulação, acessibilidade na UI.

O instrutor confirmou que a escolha é livre, desde que fundamentada.

2. Decisões do grupo e pontos em aberto

Já decidido

#	Decisão	Motivo
1	**Finalistas: EDF e prazo virtual.** As outras cinco estratégias ficam como referência.	Os dois têm chave estática (não envelhece) e custo O(log n). O instrutor deixou a escolha livre, desde que fundamentada em testes.
2	Estrutura: **heap (fila de prioridade)** com mapa id→posição, para as duas finalistas.	A ordem muda o tempo todo e só o próximo pedido interessa. Inserir, despachar, atualizar e cancelar custam O(log n); consultar o próximo custa O(1).
3	**Docas por tamanho**: Express (P), Padrão (M) e Pesada (G), um heap por classe.	Reflete a logística real (moto, van, caminhão). Elimina o bloqueio do pedido G e a necessidade de backfilling.
4	**Cenário de docas de 15 unidades: 4 Express (P), 6 Padrão (M) e 5 Pesada (G)**, com empréstimo de mão única (doca maior livre com fila vazia pega o pedido mais urgente de classes menores, nunca o contrário). Cenário mini de 8 docas (2/3/3) para depuração e diagrama. **Métricas registradas:** atraso e espera por classe (P, M, G); utilização por classe e por doca, ociosidade e nº de empréstimos; tamanho máximo e médio de cada heap; P95 e P99 do atraso; operações de heap por pedido e tempo de CPU.	Proporcional ao tempo de doca de cada classe (28% / 39% / 33%, com mix 50/35/15 e tempos 1/2/4 min). Capacidade efetiva de ~480 pedidos/h, gargalo na Express; 15 docas genéricas fariam 500/h, então a especialização custa ~4%. Num varejista desse porte a fila nunca esvazia, então a avaliação é sempre perto da capacidade (85%, 105%, 125% e varredura de 70% a 130%). O empréstimo evita ociosidade, ao custo de usar veículo grande à toa, medido pelo nº de empréstimos.
5	Tamanho vira roteamento, não critério de prioridade (só desempata).	O pacote grande não é punido na ordem.
6	**Proteger VIP é critério secundário**, não o motivo da escolha.	Decisão nova do grupo. O VIP continua medido, mas não decide sozinho.
7	Avaliação por simulação com carga perto da capacidade e várias sementes.	Em carga baixa todas empatam; a diferença só aparece no pico.

Em aberto (a validar nos testes)

- **EDF ou prazo virtual** como algoritmo final (critério na seção 5.3).
- Valor de α, se o prazo virtual continuar na disputa.
- Regra de empréstimo entre docas e se ela compensa.
- Se a especialização por tamanho vale mais que docas genéricas.

3. Estrutura comum e as duas finalistas

3.1 Ideia

Um min-heap por classe de doca. Como o prazo absoluto é um horário fixo (`prazoAbs = agora + dispatchWindow` na chegada), a chave não muda com o tempo e o heap nunca precisa ser reconstruído. As duas finalistas diferem só na chave, e uma é caso particular da outra:

	EDF	Prazo virtual
Chave	`prazoAbs`	`prazoAbs − α·score`
Relação	Prazo virtual com **α = 0**	EDF com bônus de score
Desempate	maior score, depois chegada	menor tamanho, depois chegada
Parâmetros	nenhum	α (minutos por ponto de score)
Efeito do score	nenhum na ordem principal	Com α = 1, score 90 passa na frente de score 10 se o prazo vencer menos de 80 min depois do outro

Consequência prática: varrer α de 0 em diante na simulação mostra, em uma única curva, o quanto cada ponto de bônus de score custa em atraso geral e ganha em VIP. Se a curva for quase plana, o EDF basta.

3.2 Garantia contra starvation

EDF: um pedido nunca é ultrapassado por quem tem prazo posterior; a espera é limitada pelo próprio prazo. **Prazo virtual:** um pedido A só pode ser ultrapassado por pedidos com prazo menor que `prazoA + 100·α`, então a espera também é limitada, sem contador de aging. As garantias valem dentro de cada heap.

3.3 Pseudocódigo (parametrizado por α; EDF é α = 0)

```
chave(p) = (p.prazoAbs - α*p.score, desempate, p.chegada)   # estática
fila[classe]  # um heap por classe: EXPRESS(P), PADRAO(M), PESADA(G)

inserir(p):    p.prazoAbs = agora + p.dispatchWindow
               c = classe(p.tam); fila[c].push(chave(p)); mapa[p.id] = (c, pos)   # O(log n)

proximo(c):    return fila[c].raiz                                                # O(1)

despachar(doca d):
    c = classe(d)
    se fila[c] nao vazia: return fila[c].pop()                                    # O(log n)
    senao: # empréstimo de mão única
        t = o de menor chave entre as raízes das classes menores que c            # O(1) por
classe
        se existe t: return fila[classe(t)].pop()

atualizar(id, novoScore, novoPrazo):
    (c, i) = mapa[id]; recalcula a chave; siftUp(i) ou siftDown(i)                # O(log n)

cancelar(id):  (c, i) = mapa[id]; troca com o último; remove;
               siftUp(i) ou siftDown(i) conforme o elemento trocado               # O(log n)

telaTopK(c, K): heap auxiliar que percorre o heap sem copiá-lo                    # O(K log K)
```

3.4 Complexidade

Operação	Custo
Consultar o próximo	O(1)
Inserir, despachar, atualizar, cancelar	O(log n)
Exibir os K próximos (heap auxiliar)	O(K log K). Copiar o heap custaria O(n).
Empréstimo de doca	O(1) para escolher a classe mais um pop O(log n)

3.5 Por que heap e não os outros algoritmos de ordenação

Algoritmo	Tempo médio	Pior caso	Estável	Memória	Na fila do VelozMart
Insertion Sort	$O(n^2)$	$O(n^2)$	Sim	O(1)	Encaixar cada pedido custa O(n). Lento com milhares.
Selection Sort	$O(n^2)$	$O(n^2)$	Não	O(1)	Varre a fila a cada despacho e não reaproveita a ordem.
Merge Sort	O(n log n)	O(n log n)	Sim	O(n)	Ótimo para lote; reordenar tudo a cada mudança é caro.
Quick Sort	O(n log n)	$O(n^2)$	Não	O(log n)	Mesmo problema do Merge. Pior caso com pivô fixo em fila quase ordenada.
Heap Sort / fila de prioridade	O(n log n)	O(n log n)	Não	O(1)	**Hipótese de trabalho** (a confirmar no teste de custo). O heap fica montado e só se extrai o topo quando a doca libera. Desempate por chegada compensa a falta de estabilidade.

Para a mesma chave, todos os algoritmos produzem a mesma ordem; o que muda é o custo. O teste de custo (seção 6) mede isso.

4. Cenário de testes

4.1 Premissas

- Mix de pedidos: **50% P, 35% M, 15% G**.
- Tempo de embalagem: P = 1 min, M = 2 min, G = 4 min. Tempo médio por pedido: 1,8 min.
- Divisão do tempo de doca: P 28%, M 39%, G 33%.
- VIP: `priorityScore ≥ 70`.

4.2 Docas do cenário base (15 docas)

Doca	Qtd	Atende	Veículo	Capacidade da classe	Chegada total (λ) que a classe suporta
Express	4	P	moto, carro pequeno	240/h	480/h
Padrão	6	M	van, utilitário	180/h	514/h
Pesada	5	G	caminhão	75/h	500/h

Gargalo: Express. **Capacidade efetiva ≈ 480 pedidos/h**. Quinze docas genéricas fariam 500/h; a diferença de ~4% é o custo da especialização.

Cenário mini (depuração e diagrama): 8 docas (2 Express, 3 Padrão, 3 Pesada), capacidade ≈ 240/h.

4.3 Cargas

Carga	% de 480/h	Pedidos/h
Normal	85%	408
Pico	105%	504
Sobrecarga	125%	600
Varredura	70% a 130%, passo de 5%	curva carga × atraso

4.4 Variações da configuração

Variação	O que mostra
Docas genéricas (15 iguais)	Baseline: vale especializar?
Sem empréstimo	Pior caso de rigidez
Empréstimo de mão única (recomendado)	Aproveitamento sem tirar vaga de classe maior
Mix deslocado 33/33/33 (como o mock)	Com 4/6/5 o limite da classe G cai para ~225 pedidos/h, menos da metade da capacidade-base. Mostra o risco de dimensionar errado.
Rajada de G (promoção)	Desbalanceamento temporário entre filas

4.5 Parâmetros da simulação

- Chegadas: processo de Poisson com picos ao longo do dia (nunca fila vazia).
- Pelo menos 10 mil pedidos por rodada; primeira hora descartada como aquecimento.
- 30 a 50 sementes por cenário; média e intervalo de confiança.
- Janelas de prazo: 30 a 240 min. Mesma lista de pedidos para todas as estratégias na mesma semente.

- Sementes de calibração separadas das de avaliação.
- Atraso = tempo entre o fim da embalagem e o prazo, quando positivo.

5. Estratégias a serem testadas

5.1 Finalistas e referências

Finalistas (foco da análise): EDF e prazo virtual. As outras cinco são referências: FIFO é o baseline obrigatório da pergunta III; peso composto, duas filas, faixas+aging e WSPT mostram o que acontece com outras ideias. Todas usam as mesmas filas por classe de doca, a mesma regra de empréstimo e o mesmo conjunto de pedidos; só a chave muda. Exemplo comum: A (score 30, vence em 90 min, P), B (70, 55, G), C (20, 20, P). O EDF escolhe C; o prazo virtual com $\alpha = 1$ escolhe B.

Estratégia	Chave (menor primeiro, salvo indicação)	Chave muda com o tempo?	Escolhe no exemplo	Ponto forte	Ponto fraco	Parâmetros
FIFO	ordem de chegada	Não	A	Simples e sem custo	Urgente fica atrás de tranquilo	nenhum
EDF (finalista)	prazo absoluto (desempate: maior score, chegada)	Não	C	Cauda curta, simples	Ignora importância (VIP com prazo folgado)	nenhum
Peso composto	score ÷ minutos restantes (maior primeiro)	Sim	B	Mistura importância e urgência	Nota muda a cada minuto; score baixo com prazo longo é esquecido	nenhum
Duas filas	urgentes (vence em até 60 min) por maior score; normais por chegada	Sim (migração)	B	Fácil de explicar; protege VIP	Limite rígido de 60 min; normais podem esperar muito	limite 60 min
Faixas + aging	faixa de 30 min; dentro dela, maior (score + 0,2·minutos de espera); depois menor tamanho	Sim (re-bucketing)	C	Protege VIP em sobrecarga	Efeito de borda entre faixas; dois parâmetros	faixa 30 min, aging 0,2 pt/min
WSPT	score ÷ tempo de embalagem (maior primeiro)	Não	A	Poucos atrasados no total	Ignora prazo; G de score baixo é esquecido; cauda de centenas de minutos	nenhum
Prazo virtual (finalista)	prazoAbs − α·score	Não	B	Chave estática, VIP protegido, cauda controlada, sem starvation	Atrasa um pouco mais que o EDF; exige calibrar α	α

Calibração de α. Com janelas de 30 a 240 min e score de 0 a 100, α = 1 dá bônus máximo de 100 min. Teste uma grade (por exemplo 0; 0,5; 1; 2; 3; 5) em sementes separadas da avaliação. α = 0 é o EDF e serve de controle: o prazo virtual só vale a pena se algum α > 0 melhorar claramente o EDF no que importa.

5.2 O duelo EDF × prazo virtual: o que se espera

Critério	EDF	Prazo virtual
Simplicidade e manutenção	Máxima: sem parâmetros	Um parâmetro (α) a calibrar com dados reais
% de pedidos atrasados	Tende a ser menor ou igual	Tende a atrasar um pouco mais
Cauda (P95, P99, pior atraso)	Tende a ser a menor	Maior que o EDF, mas controlada
Pedidos VIP atrasados	Tratados como qualquer outro	Tende a proteger melhor (critério secundário)
Throughput	Deve ficar praticamente igual: a doca não fica ociosa em ambos	

Essas são hipóteses. Resultados anteriores (docas genéricas) indicaram EDF com menos pedidos atrasados e cauda de 5,9 min contra 22,4 min do prazo virtual, e o prazo virtual com 0,3% de VIP atrasado contra 4,9% do EDF. A simulação nova com docas por tamanho e janelas de 30 a 240 min dirá se isso se mantém.

5.3 Critério de decisão (proposta, definir antes de rodar)

1. Métricas principais, em ordem: **% de atrasados**, **atraso médio**, **P99/pior atraso** e **throughput**.
2. O VIP entra como desempate: vale se o ganho for grande.
3. Regra sugerida: **fica o EDF**, a menos que algum $\alpha > 0$ reduza os VIPs atrasados de forma relevante (por exemplo, pela metade ou mais) **sem** piorar a % de atrasados em mais de 1 ponto percentual nem o P99 em mais de 20%. Os limites 1 pp e 20% são sugestões para o grupo ajustar antes dos testes.
4. Usar intervalos de confiança entre as sementes: diferenças dentro do intervalo contam como empate, e empate vai para o EDF pela simplicidade.
5. Resultado em todas as cargas (normal, pico, sobrecarga) e nas variações de docas, não só no pico.

Resultados anteriores (docas genéricas, 2 docas, 1.500 pedidos × 50 rodadas, pico ~105%)

Estratégia	% atrasados	% VIP atrasados	Atraso médio (min)	Pior atraso (min)
FIFO	18,4	48,3	5,3	70,6
EDF	5,2	4,9	0,5	5,9
Peso composto	4,7	3,0	0,6	46,7
Duas filas	4,2	0,0	7,8	431,6
Faixas + aging	4,9	0,0	1,8	58,3
WSPT	4,1	0,3	13,1	926,6
Prazo virtual	6,3	0,3	0,9	22,4

Estes números valiam para o cenário anterior (docas genéricas, janelas e pesos diferentes) e serão refeitos. Servem de referência de comportamento esperado.

6. Métricas, gráficos e testes de custo

6.1 Métricas coletadas

Métricas **principais** para a decisão: % atrasados, atraso médio, P99 e pior atraso, throughput.
Secundárias: VIP e as demais.

Grupo	Métricas
Atraso (principal)	% atrasados, atraso médio, P95, P99, pior atraso
VIP (secundária; score ≥ 70)	% de VIPs atrasados
Por classe	Atraso e tempo de espera de P, M e G separados
Docas	Utilização por classe e por doca, ociosidade, nº de empréstimos (doca grande atendendo pedido menor)
Fila	Tamanho máximo e médio de cada heap ao longo do tempo
Vazão	Pedidos por hora (throughput)
Custo do algoritmo	Operações de heap por pedido, tempo de CPU por operação

6.2 Gráficos planejados

1. Barras agrupadas: estratégia × % atrasados em carga normal, pico e sobrecarga.
2. Curva carga × % atrasados (70% a 130%), uma linha por estratégia.
3. Boxplot por semente do atraso médio e do P99.
4. CDF do atraso, para mostrar a cauda.
5. Série temporal do tamanho das filas P, M e G no pico.
6. Mapa de calor da utilização por doca e por carga.
7. Pareto: % VIP atrasado × pior atraso.
8. Curva de α (0 = EDF): % atrasados, P99 e % VIP atrasado em função de α, para ver o custo e o ganho do bônus de score.
9. Cenário de docas genéricas × especializadas (com e sem empréstimo).

6.3 Teste de custo dos algoritmos de ordenação

Mesma chave (prazo virtual) implementada com Insertion, Selection, Merge, Quick e Heap, reordenando a cada mudança versus manter o heap. Medir tempo por inserção, despacho e atualização para n = 1 mil, 10 mil e 100 mil pedidos. Esperado: Heap O(log n) por operação contra O(n) ou O(n log n) dos demais.

7. Dados sintéticos (mock existente e ajustes)

Arquivos do grupo: `build_mock.mjs` (gerador, semente 20261110) e `mock_dados_ordenacao_prioridade.xlsx` (abas Parametros, Produtos com 45 itens e Pedidos com 90 pedidos). O score usa distância 30%, SLA 30%, valor 20% e reputação 20%. O mock serve para ilustrar o formato e o diagrama, mas não para provar o algoritmo.

#	Problema no mock	Ajuste
1	SLA de 137 a 1440 min com α = 1: o score quase nunca decide.	Janelas de 30 a 240 min no gerador de carga e α calibrado.
2	Três pedidos por dia: não existe fila, todas as estratégias empatam.	Novo gerador de carga com Poisson e picos (seção 4).
3	Pesos do score diferem do documento antigo (40/20/25/15).	Alinhar um conjunto de pesos nos dois lugares.
4	Mix uniforme (33/33/33) contra 50/35/15 do documento.	Usar 50/35/15 no teste base; manter 33/33/33 como variação.
5	`Parametros` só tem `numero_docas = 2`.	Adicionar docas por classe e regra de empréstimo.
6	`dispatchWindow` depende do relógio fixo (01/11) e vai a 43 mil min.	Simular com `prazo_expedicao` absoluto; calcular a janela só na exibição.
7	Prazo de expedição = chegada + SLA inteiro.	Declarar como simplificação.
8	Tamanho independente de valor e distância.	Opcional: correlacionar valor ao tamanho.
9	Caminho de saída do Mac e dependência `@oai/artifact-tool`.	Trocar o caminho e confirmar a dependência.

8. Correções pendentes no documento principal

- Renomear o título: "Faixas, Aging e Proteção de Doca" não descreve o algoritmo. Sugestão: **"VelozMart MPS: Fila de Expedição por Prazo (EDF e Prazo Virtual) com Docas por Tamanho"**. Depois da simulação, usar o nome do algoritmo escolhido.
- Reescrever a justificativa: o documento antigo escolheu o prazo virtual para proteger VIPs. A nova justificativa sai do critério da seção 5.3.
- Trocar o Top-K por O(K log K) com heap auxiliar (copiar o heap custa O(n)).
- Substituir backfilling e reserva de vaga pelo modelo de docas por tamanho e refazer os trade-offs.
- Completar o `cancelar` com siftUp/siftDown após a troca.
- Chamar a estrutura de "heap / fila de prioridade" e corrigir a frase "o maior sobe ao topo" (é um min-heap).
- No Quick Sort, escrever que o pior caso em fila quase ordenada vale para pivô fixo.
- Não declarar vencedor antes dos testes. Faixas+aging teve menos atrasados e 0% de VIP atrasado no pico anterior, então também não pode ser ignorada no texto.
- Declarar que os concorrentes tiveram parâmetros assumidos (ou calibrá-los do mesmo modo) e o viés do score gerado a partir do SLA.
- Informar média e intervalo de confiança das rodadas; definir "atraso médio" (todos os pedidos ou só os atrasados).

- Conferir o alinhamento das tabelas de resultados e do mapa de trade-offs no PDF final.
- Entrega: cabeçalho com Grupo, Turma e Prazo; formato Canvas em PDF.

9. Trade-offs esperados (rascunho para o Canvas)

Rascunho; será confirmado pelos números da simulação.

EDF

Prós	Contras
1. Sem parâmetros: simples de explicar, implementar e manter.	1. Ignora importância: um VIP com prazo folgado espera como os demais.
2. Chave estática; O(1) para consultar e O(log n) para o resto.	2. Não resolve sobrecarga sustentada; falta capacidade, não regra.
3. Cauda de atraso tende a ser a menor entre as estratégias.	3. Docas por tamanho podem ficar desbalanceadas, e o EDF não corrige isso.

Prazo virtual

Prós	Contras
1. Chave estática, como o EDF, sem re-bucketing.	1. Tende a atrasar um pouco mais pedidos que o EDF.
2. Considera o score, protegendo VIPs sem abandonar pedidos.	2. Exige calibrar α com dados reais.
3. Um parâmetro com leitura de negócio e limite explícito de ultrapassagem (100·α).	3. Mais difícil de explicar e de manter que o EDF; como o VIP pesa menos, o ganho pode não compensar.

Mapa simplicidade × desempenho × manutenibilidade

Será preenchido com os números da nova simulação. Hipótese atual: FIFO e EDF têm simplicidade alta e desempenho baixo/médio; peso composto e faixas+aging, simplicidade média e manutenção média; WSPT e duas filas, bom em atrasados mas cauda grande; prazo virtual, simplicidade e manutenibilidade altas e desempenho alto no pico com cauda curta. O EDF deve ter simplicidade e manutenibilidade máximas e a menor cauda. O resultado da simulação decide.

10. Próximos passos

1. Alinhar pesos do score, mix e janelas; atualizar o `build_mock.mjs` ou criar o gerador de carga.
2. Implementar o simulador (eventos discretos) com as 7 estratégias, docas por classe e empréstimo.
3. Fixar o critério de decisão (seção 5.3) **antes** de rodar os testes.
4. Calibrar α (e, igualmente, os parâmetros dos concorrentes) em sementes separadas.
5. Rodar cenários e variações, coletar métricas e gerar os gráficos.
6. Rodar o teste de custo dos 5 algoritmos de ordenação.
7. **Escolher o algoritmo final** (EDF ou prazo virtual) aplicando o critério.
8. Refazer o documento principal com o algoritmo escolhido, os resultados novos e as correções da seção 8.
9. Propor teste A/B em região piloto (prazo virtual contra EDF: atraso médio e % VIP atrasado).

10. Interface: indicar urgência por cor **e** texto, nunca só por cor.
11. Futuro: aprendizado online do α, previsão de demanda por classe e realocação dinâmica de docas.